Testing and QA Summary

Status date: April 29, 2026

This document explains *what* is tested, *how* it is tested, and *what is not covered yet*. It is the QA contract for the next developer.

1. How to run the suite

From the repository root:

```
flutter pub get
./scripts/check_platform_gating.sh   # platform-import gate (fast)
flutter analyze                       # static analysis (lints + unused, etc.)
flutter test                          # 21 test files (excluding test_util.dart)
flutter build web --release           # production build sanity check
```

The first three validation commands after `flutter pub get` match the checks in `.github/workflows/test.yml`; the deploy workflow (`.github/workflows/deploy.yml`) re-runs the release web build. A change that breaks any of them must not be merged to `main`.

To run a single test file:

```
flutter test test/integration_workflow_test.dart
```

2. Test inventory

The repository contains **21 Dart test files** plus one shared `test_util.dart` helper, organized by what they exercise:

Parsing and stores

File	Covers
`courses_test.dart`	`Courses.loadCourses` / `loadCoursesFromBytes`: tab parsing, whitespace tolerance, duplicate codes (`DuplicateCourseCodeException`), malformed-row detection.
`people_test.dart`	`People.loadPeople` / `loadPeopleFromBytes`: 21-column parse, availability encoding (`'', '1', '2', '3'`), duplicate selection, listing-when-wanting-zero, wanting-more-than-listed, malformed inputs, missing-submission default.
`validate_test.dart`	People-vs-courses consistency (`InconsistentCourseAndPeopleException`).
`auxiliary_data_test.dart`	Helper data structures used across compute modules.

Compute modules

File	Covers
`course_control_test.dart`	Drop, split-mode (`split` vs `limit`), per-course min/max overrides, coordinator data structures.
`split_control_test.dart`	Split preview vs commit, John Taber's algorithm correctness, history-aware resplitting.
`overview_data_test.dart`	First choices, backups, drops, resulting class membership, unmet-wants summary math.
`scheduling_test.dart`	The `Scheduling` facade end-to-end: state transitions, `compute(Change)` propagation, output-state gating.
`schedule_unavailability_update_test.dart`	Recomputation of class-size & availability counts after scheduling and drop events (the D5 fix).

Save / load / persistence

File	Covers
`import_export_test.dart`	`exportStateToString` / `loadState` round-trip, including all sections of the state grammar.
`bundled_state_test.dart`	`buildBundledStateContent` / `parseBundledStateContent`: the `CourseFile:` + `PeopleFile:` + `Setting:` envelope, missing-marker errors, legacy state-only files, no-trailing-newline edge cases, CRLF content, round-trip build→parse.
`save_load_compatibility_test.dart`	Backwards compatibility between bundled saves and legacy state-only saves; restore preserves drops, splits, schedule, coordinators; no-trailing-newline sources; adjacent-marker legacy saves; CRLF sources; coordinator-state cross-instance restore.
`autosave_store_web_test.dart`	Browser `localStorage` keys (`omnilore_autosave`, `omnilore_hardsave`, `*_time`, `omnilore_course_data`, `omnilore_people_data`); set/clear/timestamp behavior.
`export_text_file_test.dart`	`exportTextFile()` helper: web `showSaveFilePicker` path, fallback download path, desktop `pickSavePath`/`writeToPath` path, cancel/empty-path no-op.

Widget / UI

File	Covers
`coordinator_widget_test.dart`	`Set C and CC` and `Set CC1 and CC2` clearing-on-reentry behavior; tap-to-clear an assigned coordinator.
`class_name_display_coordinator_mode_test.dart`	Coordinator-mode UI states on the course rows.
`overview_data_widget_test.dart`	Overview/stat panel rendering across pipeline stages.
`schedule_deselect_widget_test.dart`	Click-same-slot-again deselects (D6 feature).
`integration_workflow_test.dart`	End-to-end pipeline: load → drop → split → schedule → coordinator → export. The single richest regression test.

Platform safety

File	Covers
`platform_gating_test.dart`	Mirrors `scripts/check_platform_gating.sh` at test time, ensuring `dart:io` and desktop-only imports stay outside the allowlist.

Shared helpers

File	Covers
`test_util.dart`	`hasMessage(String)` custom matcher used to assert exception messages without brittle string equality.

3. What was tested across deliverables (cumulative)

The grid below summarizes tests that were added or stabilized in each deliverable cycle. Source: D4, D5, and D6 final reports.

Capability	First validated	Status at D7
Course / people parsing and malformed-input detection	D2/D3	Stable; covers all exception types in `lib/model/exceptions.dart`.
Web-safe file loading via byte streams	D3/D4	Stable; both desktop and web paths covered.
Browser file picker + downloads	D4	Stable; `export_text_file_test.dart` covers all four web/desktop branches.
Save / load intermediate state	D4	Stable; covered by `import_export_test.dart` and `save_load_compatibility_test.dart`.
Bundled state (CourseFile + PeopleFile + Setting)	D4/D5	Stable; covered by `bundled_state_test.dart`. Legacy-autosave recovery (CourseFile without PeopleFile) covered by trial-run patch `70889a9`.
Autosave / hardsave in `localStorage`	D4	Stable; `autosave_store_web_test.dart`.
Coordinator workflow (`Set C and CC`, `Set CC1 and CC2`, equal export labels)	D4/D5	Stable; `coordinator_widget_test.dart`, `class_name_display_coordinator_mode_test.dart`, integration.
Recomputation after scheduling/drop changes	D5	Stable; `schedule_unavailability_update_test.dart`.
Schedule deselection	D6	Stable; `schedule_deselect_widget_test.dart`.
Filename-on-export consistency (early/final/mail-merge/unmet-wants)	D6	Stable; `export_text_file_test.dart`.
Platform gating	D3 onwards	Stable; both grep script and Dart test.

4. Numbers (as of last on-machine run)

- **Test files:** 21 (+ `test_util.dart`).
- **Tests passing:** 99/99 on the latest verification run (branch `codex/fix-state-load`, commit `70889a9`, 2026-04-30). The PR #11 post-D7 bundled-state fix added 8 tests (98/98); a subsequent trial-run patch added 1 more for the legacy-autosave recovery path (99/99). D7 baseline was 90/90 at commit `8a38334`.
- **Static analysis:** 0 issues.

- **Web release build:** passes with non-blocking advisories only (Flutter service-worker / Wasm notes).
- **Coverage tooling:** not configured. Adding `flutter test --coverage` and an LCOV report is filed as P2 in the backlog.

5. Manual QA Script (run against the live URL after every deploy)

Use `test/resources/course.txt` and `test/resources/people.txt` unless the stakeholder provides current term data.

1. Open http://scheduler.omnilore.org. Verify the home screen renders without console errors (`Cmd-Opt-J` in Chrome).
2. **File → Import Course →** `course.txt`. State indicator becomes `needPeople`.
3. **File → Import People →** `people.txt`. Overview/stats panel populates.
4. Confirm at least one class is flagged as undersize and at least one as oversize. Otherwise, set a custom `min`/`max` to force one of each.
5. Drop one undersize class. Choose `Limit` (cap-at-max) for one and `Split` (preview) for another oversize class.
6. Open **Show Splits** for an oversize class. Move at least one person between preview groups. **Cancel** once. Re-enter, then **Implement**.
7. Open the schedule grid. Assign every surviving class to a 0–19 slot. Click an assigned slot again — confirm it deselects and the class returns to unscheduled.
8. Open the coordinator panel. Assign **C / CC** on one class and **CC1 / CC2** (equal) on another.
9. Tap a highlighted name on a coordinator-assigned course; confirm the assignment clears.
10. Export each artifact. Verify each file is non-empty and named correctly:
    - `early_roster_YYYY-MM-DD_HHMM.txt` (phone-roster style early export)
    - `final_roster_YYYY-MM-DD_HHMM.txt` (with `(C)` and `(CC)`)
    - `mail_merge_YYYY-MM-DD_HHMM.txt` (tab-delimited; check at least one row has 9+ class columns)
    - `unmet_wants_YYYY-MM-DD_HHMM.txt` (per-person `Wants/Unmet/Assigned` blocks)
11. **Save As** to a bundled file. Inspect it in a text editor — confirm the three sections (`CourseFile:`, `PeopleFile:`, `Setting:`).
12. Refresh the page. Decline the autosave restore prompt. Open **Load** and pick the bundled file you just saved.
13. Confirm course list, people list, drops, splits, schedule, and coordinators all match.

6. Known QA gaps

These are explicit gaps, not unknowns:

Gap	Why	Mitigation / next step
Stakeholder-data live smoke test	A full trial run was performed on 2026-04-30 with live stakeholder data; two bugs found and fixed (commit `70889a9`).	Re-run against the deployed URL after `codex/fix-state-load` merges to `main`.
Demo video	Not yet recorded	Use `Demo_Video_Script_and_Checklist.md`. Filed P0.
Code coverage report	Not configured	Add `flutter test --coverage` + LCOV upload. P2.
Cross-browser save-dialog matrix	`showSaveFilePicker` support varies	Manual matrix across Chrome/Edge/Safari/Firefox at next QA cycle. P2.

Gap	Why	Mitigation / next step
Performance with very large datasets	Canonical fixture is 24/267	Profile load+split+schedule with 5–10× data; document p50/p95. P2.
Keyboard-only navigation	Not separately validated	Run keyboard-only QA pass; add focus traps where missing. P3.
Screen-reader labels	Many controls are icon-only	Add Semantics labels on the table and coordinator widgets. P3.

7. CI signals to watch

- **`Test` workflow** on every PR and push to main. A red run blocks merge.
- **`Deploy Web` workflow** on every push to `main`. A red run means the live site did not update — re-run from the same commit, or revert the offending commit.
- The **revoke ingress** step in `deploy.yml` runs `if: always()` so a rules table dangling after a failed deploy is rare. If it does happen, manually delete the temporary `:22` rule on `AWS_SG_ID` (Operations Runbook §6).